```
//*****************************************************************************
//*****************************************************************************
//                                                                           **
//                       PROCESOR LPC2129/01                                 **
//      funkcjonalnosci:                                                     **
//                    - implementacja testów do każdej stworzonej funkcji    **
//                                                                           **
//                                                                           **
//*****************************************************************************


void TestOf_CopyString(void) {

    char pcStandard[] = "standardowy lancuch";
    char pcEmpty[] = "";
    char pcDestination[21];

    printf("CopyString \n\n");
    printf ("Test 1 - ");
    // Kopiowanie standardowego lancucha znakow
    CopyString(pcStandard, pcDestination);
    if (EQUAL == eCompareString(pcStandard, pcDestination)) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Kopiowanie pustego lancucha znakow
    CopyString(pcEmpty, pcDestination);
    if (EQUAL == eCompareString(pcEmpty, pcDestination)) printf("OK \n"); else printf("Error \n");
}
```

```
void TestOf_eCompareString(void) {

    char pcStr[] = "lancuch znakowy";
    char pcEqualStr[] = "lancuch znakowy";
    char pcDifferentStr[] = "inny lancuch";
    enum CompResult eResult;

    printf("eCompareString \n\n");
    printf ("Test 1 - ");
    // Porównanie takich samych łańcuchów znakowych
    eResult = eCompareString(pcStr, pcEqualStr);
    if (EQUAL == eResult) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Porównanie różnych łańcuchów znakowych
    eResult = eCompareString(pcStr, pcDifferentStr);
    if (DIFFERENT == eResult) printf("OK \n"); else printf("Error \n");
}


void TestOf_AppendString(void) {

    char pcFullStr[] = "pelne zdanie";
    char pcFirstWord[13] = "pelne ";
    char pcLastWord[] = "zdanie";
    char pcUnchangedLastWord[] = "zdanie";
    char pcEmpty[] = "";

    printf("AppendString \n\n");
    printf ("Test 1 - ");
    // Dodanie dwóch wyrazów do siebie
    AppendString(pcLastWord, pcFirstWord);
    if (EQUAL == eCompareString(pcFirstWord, pcFullStr)) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Dodanie pustego łańcucha znakowego
    AppendString(pcEmpty, pcLastWord);
    if (EQUAL == eCompareString(pcLastWord, pcUnchangedLastWord)) printf("OK \n"); else printf("Error \n");
}
```

```
void TestOf_ReplaceCharactersInString(void) {

    char pcStr1[] = "Oryginalny napis";
    char pcStr2[] = "Oryginalny napis";
    char pcReplacedStr1[] = "Orygiwalwy wapis";
    char pcReplacedStr2[] = "Orygi";
    char cOldChar = 'n';
    char cNewChar = 'w';
    char cNullChar = '\0';

    printf("ReplaceCharactersInString \n\n");
    printf ("Test 1 - ");
    // Zamiana wskazanej wartości znaków na inny wskazany znak
    ReplaceCharactersInString(pcStr1, cOldChar, cNewChar);
    if (EQUAL == eCompareString(pcStr1, pcReplacedStr1)) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Zamiana wskazanej wartości znaków na znak NULL
    ReplaceCharactersInString(pcStr2, cOldChar, cNullChar);
    if (EQUAL == eCompareString(pcStr2, pcReplacedStr2)) printf("OK \n"); else printf("Error \n");
}
```

```
void TestOf_UIntToHexStr(void) {

    unsigned int uiZero = 0;
    unsigned int uiSmallNumber = 1;
    unsigned int uiBigNumber = 65000;
    char pcConvertedStr[7] = "";
    char pcHexZero[] = "0x0000";
    char pcHexSmallNumber[] = "0x0001";
    char pcHexBigNumber[] = "0xFDE8";

    printf("UIntToHexStr \n\n");
    printf ("Test 1 - ");
    // Zamiana liczby równej 0
    UIntToHexStr(uiZero, pcConvertedStr);
    if (EQUAL == eCompareString(pcConvertedStr, pcHexZero)) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Zamiana małej liczby
    UIntToHexStr(uiSmallNumber, pcConvertedStr);
    if (EQUAL == eCompareString(pcConvertedStr, pcHexSmallNumber)) printf("OK \n"); else printf("Error \n");

    printf ("Test 3 - ");
    // Zamiana dużej liczby
    UIntToHexStr(uiBigNumber, pcConvertedStr);
    if (EQUAL == eCompareString(pcConvertedStr, pcHexBigNumber)) printf("OK \n"); else printf("Error \n");
}
```

```
void TestOf_eHexStringToUInt(void) {

    char pcCorrectHex[] = "0xFA65";
    char pcBadLetterHex[] = "0xFDG5";
    char pcNoPrefixHex[] = "FDG5";
    unsigned int uiExpectedValue = 64101;
    unsigned int uiValue;
    enum Result eResult;

    printf("eHexStringToUInt \n\n");
    printf ("Test 1 - ");
    // Zamiana poprawnej liczby heksadecymanlnej
    eResult = eHexStringToUInt(pcCorrectHex, &uiValue);
    if ((uiExpectedValue == uiValue) && (OK == eResult)) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Zamiana liczby heksadecymanlnej z niepoprawną literą
    eResult = eHexStringToUInt(pcBadLetterHex, &uiValue);
    if (ERROR == eResult) printf("OK \n"); else printf("Error \n");

    printf ("Test 3 - ");
    // Zamiana liczby heksadecymanlnej bez prefiksu
    eResult = eHexStringToUInt(pcNoPrefixHex, &uiValue);
    if (ERROR == eResult) printf("OK \n"); else printf("Error \n");
}

void TestOf_AppendUIntToString(void) {

    char pcStr1[15] = "liczba: ";
    char pcExpectedStr1[] = "liczba: 0xFDE8";
    char pcStr2[15] = "liczba: ";
    char pcExpectedStr2[] = "liczba: 0x0000";
    unsigned int uiNumber = 65000;
    unsigned int uiZero = 0;

    printf("AppendUIntToString \n\n");
    printf ("Test 1 - ");
    // Dodanie liczby do łańcucha znakowego
    AppendUIntToString(uiNumber, pcStr1);
    if (EQUAL == eCompareString(pcStr1, pcExpectedStr1)) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Dodanie zera do łańcucha znakowego
    AppendUIntToString(uiZero, pcStr2);
    if (EQUAL == eCompareString(pcStr2, pcExpectedStr2)) printf("OK \n"); else printf("Error \n");
}
```

```
void TestOf_ucFindTokensInString(void) {

    char pcStrWithMoreDelimitersBetweenTokens[] = "abc  def";
    char pcStrWithDelimiterFirst[] = "   abc";
    char pcEmptyStr[] = "  ";
    char pcStrWithExceededTokenNr[] = "ab ab ab ab";
    unsigned char ucTokenCount;

    printf("ucFindTokensInString \n\n");
    printf ("Test 1 - ");
    // Sprawdzenie wiadomości z większą ilością delimiterów między tokenami
    ucTokenCount = ucFindTokensInString(pcStrWithMoreDelimitersBetweenTokens);
    if (2 == ucTokenCount) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // Sprawdzenie wiadomości z delimiterami na początku
    ucTokenCount = ucFindTokensInString(pcStrWithDelimiterFirst);
    if (1 == ucTokenCount) printf("OK \n"); else printf("Error \n");

    printf ("Test 3 - ");
    // Sprawdzenie pustej wiadomości
    ucTokenCount = ucFindTokensInString(pcEmptyStr);
    if (0 == ucTokenCount) printf("OK \n"); else printf("Error \n");

    printf ("Test 4 - ");
    // Sprawdzenie wiadomości, przekraczającej maksymalną ilość tokenów
    ucTokenCount = ucFindTokensInString(pcStrWithExceededTokenNr);
    if (3 == ucTokenCount) printf("OK \n"); else printf("Error \n");
}
```

```
void TestOf_eStringToKeyword(void) {

    char pcLoad[] = "load";
    char pcStore[] = "store";
    char pcReset[] = "reset";
    char pcIncorrect[] = "abc";
    enum KeywordCode eKeywordCode;
    enum Result eResult;

    printf("eStringToKeyword \n\n");
    printf ("Test 1 - ");
    // sprawdzenie dla łańcucha znaków o wartości "load"
    eResult = eStringToKeyword(pcLoad, &eKeywordCode);
    if ((LD == eKeywordCode) && (OK == eResult)) printf("OK \n"); else printf("Error \n");

    printf ("Test 2 - ");
    // sprawdzenie dla łańcucha znaków o wartości "store"
    eResult = eStringToKeyword(pcStore, &eKeywordCode);
    if ((ST == eKeywordCode) && (OK == eResult)) printf("OK \n"); else printf("Error \n");

    printf ("Test 3 - ");
    // sprawdzenie dla łańcucha znaków o wartości "reset"
    eResult = eStringToKeyword(pcReset, &eKeywordCode);
    if ((RST == eKeywordCode) && (OK == eResult)) printf("OK \n"); else printf("Error \n");

    printf ("Test 4 - ");
    // sprawdzenie dla łańcucha znaków, która nie jest poprawnym słowem kluczowym
    eResult = eStringToKeyword(pcIncorrect, &eKeywordCode);
    if (ERROR == eResult) printf("OK \n"); else printf("Error \n");
}
```

```
void TestOf_DecodeTokens(void) {

    char pcKeywaord[] = "store";
    char pcNumber[] = "0x0001";
    char pcStr[] = "abc";
    ucTokenNr = 3;

    asToken[0].uValue.pcString = pcKeywaord;
    asToken[1].uValue.pcString = pcNumber;
    asToken[2].uValue.pcString = pcStr;

    printf("DecodeTokens \n\n");
    printf ("Test 1 - ");
    // Sprawdzenie czy poprawnie określono typy tokenów
    DecodeTokens();
    if ((KEYWORD == asToken[0].eType) && (ST == asToken[0].uValue.eKeyword) && (NUMBER == asToken[1].eType) &&
       (1 == asToken[1].uValue.uiNumber) && (STRING == asToken[2].eType) && (pcStr == asToken[2].uValue.pcString))
       printf("OK \n"); else printf("Error \n");
}


void TestOf_DecodeMsg(void) {

    char pcMsg[] = "reset 0x10 test";
    enum KeywordCode eExpectedKeywordCode = RST;
    unsigned int uiExpectedNumber = 16;
    char pcExpectedStr[] = "test";

    printf("DecodeMsg \n\n");
    printf ("Test 1 - ");
    // Sprawdzenie czy poprawno zdekodowano wiadomość
    DecodeMsg(pcMsg);
    if((3 == ucTokenNr) && (KEYWORD == asToken[0].eType) && (eExpectedKeywordCode == asToken[0].uValue.eKeyword) &&
       (NUMBER == asToken[1].eType) && (uiExpectedNumber == asToken[1].uValue.uiNumber) && (STRING == asToken[2].eType) &&
       (EQUAL == eCompareString(pcExpectedStr, asToken[2].uValue.pcString))) printf("OK \n"); else printf("Error \n");
}
```

```
void main(void) {

    printf("TESTY FUNKCJI DO OPERACJI NA STRINGACH \n\n\n");

    TestOf_CopyString();
    TestOf_eCompareString();
    TestOf_AppendString();
    TestOf_ReplaceCharactersInString();

    printf("TESTY FUNKCJI DO KONWERSJII \n\n\n");

    TestOf_UIntToHexStr();
    TestOf_eHexStringToUInt();
    TestOf_AppendString();

    printf("TESTY FUNKCJI DO DEKODOWANIA KOMUNIKATÓW \n\n\n");

    TestOf_ucFindTokensInString();
    TestOf_eStringToKeyword();
    TestOf_DecodeTokens();
    TestOf_DecodeMsg();
}
```